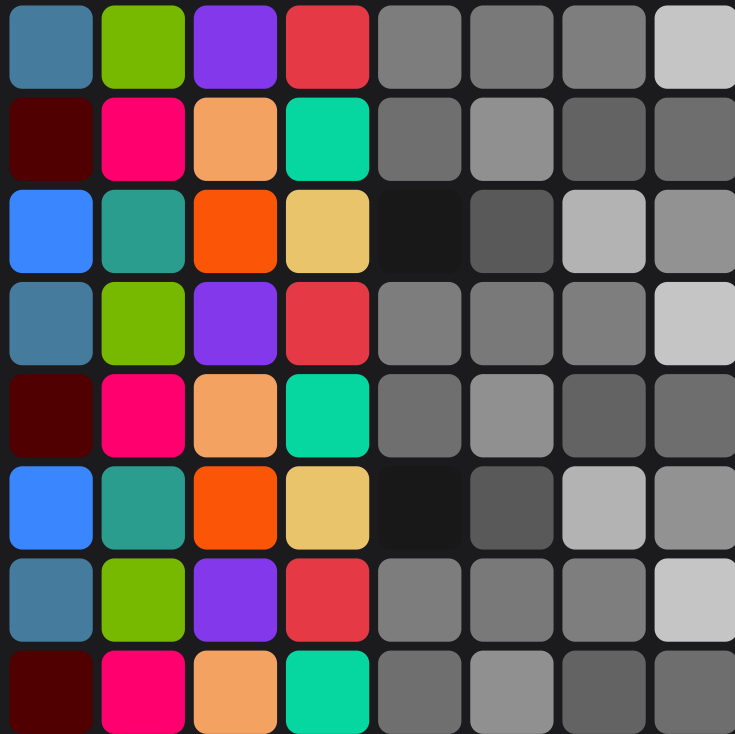


AGGIE CODING CLUB

CUDA Vision

Learning GPU programming, one pixel
at a time

Beginner-friendly · No GPU required · Basic C++ is plenty



What is CUDA Vision?

A semester-long learning project for anyone curious about how GPUs work.

If you have written a for-loop in C++ (or honestly any language), you are ready. We start from zero on GPUs and build up together, one small program at a time.



Learn how GPUs think

Parallel programming explained with pictures, not proofs.



Build real image filters

Every exercise produces an image you can actually see.



No GPU? No problem

We use free cloud GPUs, so any laptop works.

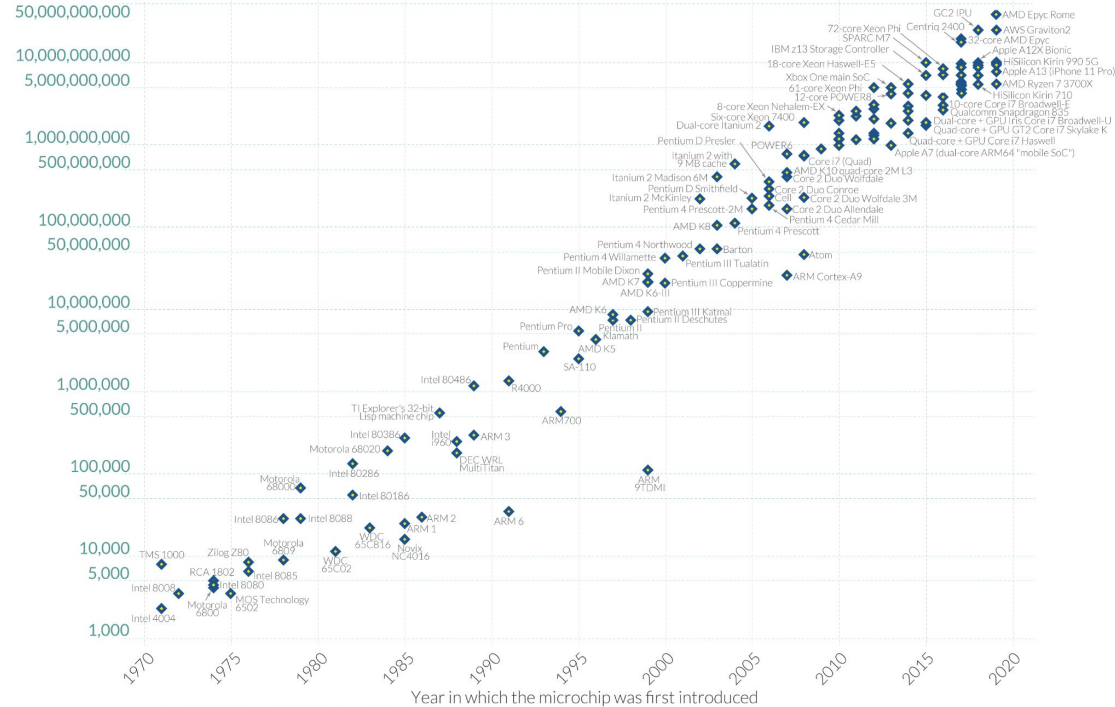
Background

Moore's Law: The number of transistors on microchips has doubled every two years

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World
in Data

Transistor count



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)

OurWorldinData.org – Research and data to make progress against the world's largest problems.

Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

Dennard Scaling & Growth

Dennard Scaling

- Power requirements proportional to area.
- Smaller transistors result in same power density.

More performance

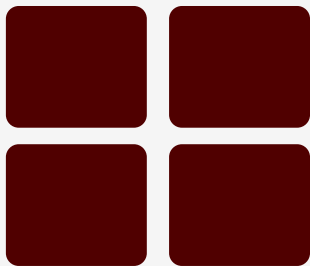
- Higher clock speeds (higher power consumption)
- At very small scales, Dennard Scaling breaks down

How do we still get more performance?

Parallelization: CPU vs GPU

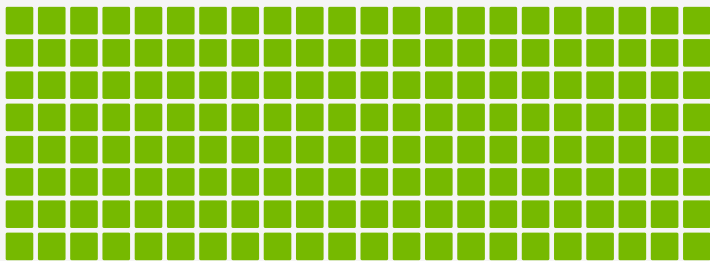
More processors can do more work.

CPU



A few very powerful and capable cores. Great at complex tasks done one after another.

GPU



Thousands of simpler cores. Great at doing the same simple job to lots of data at once.

So what is CUDA?

CUDA is NVIDIA's platform that lets you write C++ code that runs on the GPU.

2007

Year NVIDIA released CUDA

C++

Plus a few extra keywords

1000s

Threads running your code at once

CUDA only runs on NVIDIA GPUs, which is why will use Google Colab and other sites that can run it for free.

The big picture: where CUDA shows up



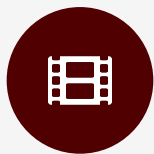
AI & deep learning

PyTorch and TensorFlow use CUDA to train and run neural networks.



Science & simulation

Weather models, molecular dynamics, physics simulations.



Graphics & video

Rendering, video editing, and image processing.



Robotics & self-driving

Turning camera feeds into decisions in real time.

Why it matters for your career

NVIDIA

Created CUDA. It is the core of their AI and graphics software.

AMD

HIP / ROCm: code looks nearly identical to CUDA, so the skills transfer directly.

Intel, Apple & more

SYCL, Metal and others use the same parallel ideas. So do Google, Meta, Tesla and AI startups.

Skills you can put on a resume

- C++ and working with memory directly
- Parallel programming (thinking in threads)
- Measuring and improving performance
- Image processing / computer vision basics
- A finished project you can show and explain

Example resume line: *"Wrote CUDA C++ kernels for image filters (grayscale, blur) and measured speedup over a CPU version of X%."*

Can be considered a “kernel” as you are compiling it uniquely with CUDA, nvcc.

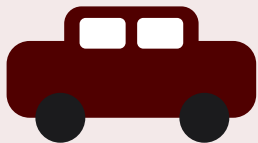
PART 2 · LEARNING

GPU code that looks like the C++ standard library

We let a library do the hard part: Thrust.

- 1 When is a GPU actually faster?
- 2 How C++ becomes GPU instructions
- 3 The CUDA library stack
- 4 Thrust: std::, GPU edition
- 5 Host vs device: policies & specifiers
- 6 3 hands-on exercises

Faster at what? A car and a bus



CPU = the car

Optimized for latency

Gets **a few** people there fast.

Takes many trips to move a crowd.

Few, powerful cores. Low overhead to get started.



GPU = the bus

Optimized for throughput

Slower for a few people.

Moves **a crowd** in one trip.

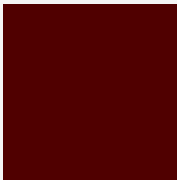
Thousands of simple cores. Costs time to "load up".

"Which is faster?" is the wrong question. Ask: **faster at moving how much?**

Problem size picks the winner

SMALL JOB

512 elements

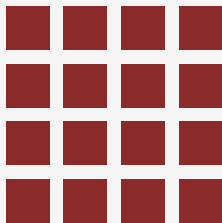


Fastest: **1 CPU thread**

Starting extra threads, or waking up the GPU, costs more time than the work itself.

MEDIUM JOB

65,536 elements

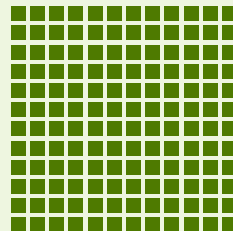


Fastest: **64 CPU threads**

Now there is enough work to split across CPU cores, so multithreading pays off. Multithreading breaks up processes into pieces to then be run at the same time.

HUGE JOB

268 million elements

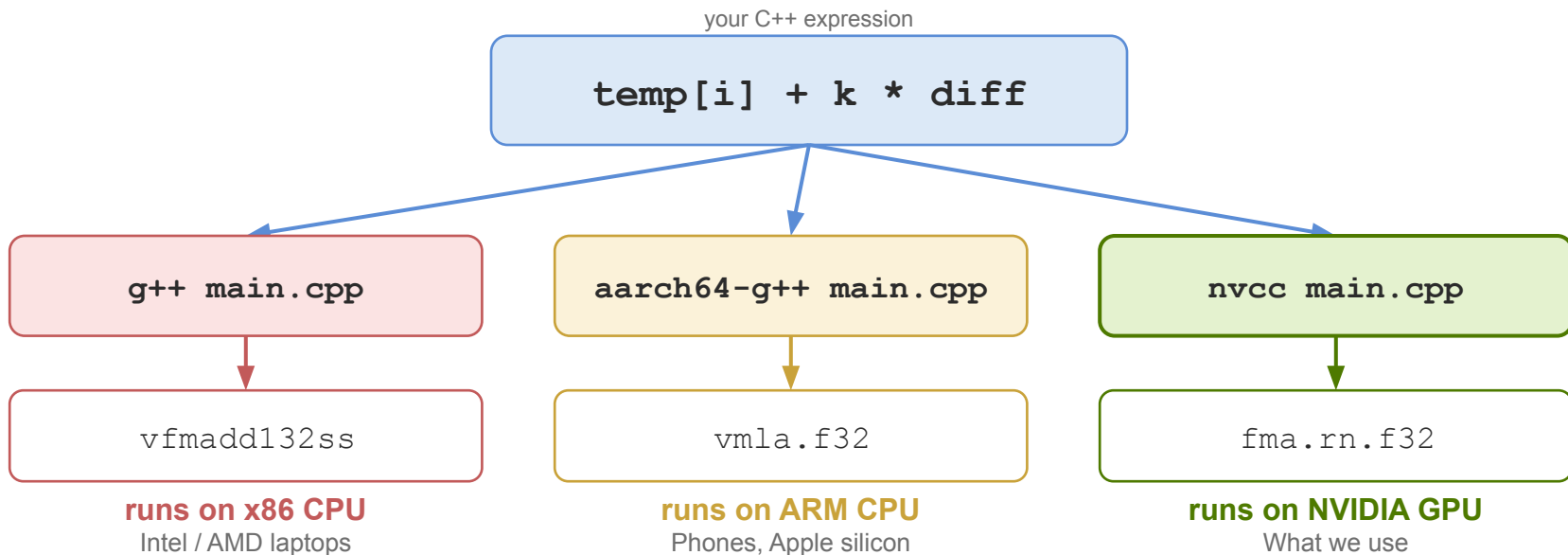


Fastest: **GPU**

The setup cost is tiny next to the work. Thousands of GPU threads win easily.

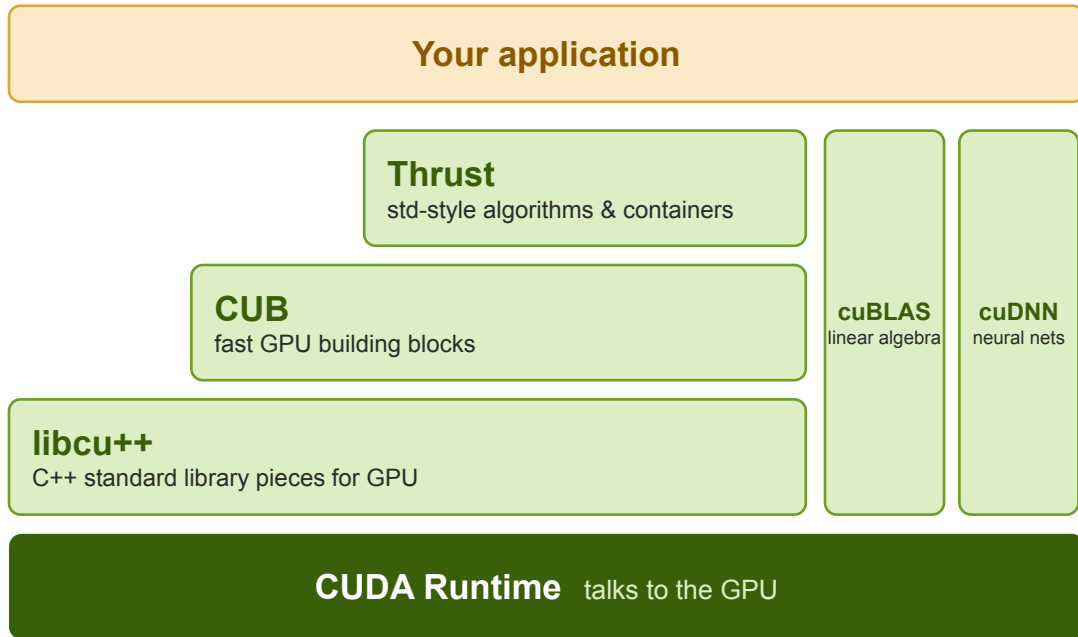
Takeaway: the GPU only wins when you give it enough work. Images (millions of pixels) are a perfect fit.

One line of C++, three different machines



nvcc (NVIDIA CUDA Compiler) knows how to turn C++ into GPU instructions. Same source, new target.

The CUDA software stack



Why use libraries?

Simpler code: one call instead of a hand-written kernel.

Faster code: tuned by NVIDIA engineers for every GPU.

Today: we live at the Thrust layer.

Underneath it all, the CUDA Runtime moves data and launches work on the GPU.

Thrust: the standard library, GPU edition

If you know `std::`, you already know most of `thrust::`

	C++ standard library	Thrust (runs on the GPU too)
Containers	<code>std::vector</code>	→ <code>thrust::host_vector</code> · <code>device_vector</code> · <code>universal_vector</code>
Algorithms	<code>std::copy</code> / <code>sort</code> / <code>transform</code>	→ <code>thrust::copy</code> / <code>sort</code> / <code>transform</code>
Extended	<i>(no <code>std::</code> version)</i>	→ <code>thrust::tabulate</code> · <code>sort_by_key</code> · <code>reduce_by_key</code>
Function objects	<code>std::plus</code> / <code>less</code> / <code>multiplies</code>	→ <code>thrust::plus</code> / <code>less</code> / <code>multiplies</code>
Iterators	<i>(no <code>std::</code> version)</i>	→ <code>thrust::counting_iterator</code> · <code>transform_iterator</code>

Full reference: nvidia.github.io/cccl/thrust

Execution policy: tell Thrust where to run

Standard C++

```
std::transform(temp.begin(), temp.end(),  
               temp.begin(), op);
```

CUDA C++ with Thrust

```
thrust::transform(thrust::device,  
                  temp.begin(), temp.end(),  
                  temp.begin(), op);
```

Only change: one extra first argument

thrust::host

→ **CPU (host)** Runs the algorithm on the CPU.

thrust::device

→ **GPU (device)** Runs the algorithm on the GPU.

Same interface on both sides, so the same code can run on the CPU or the GPU by swapping one word.

Policy vs specifier: two different jobs

Execution policy

`thrust::host` / `thrust::device`

Decided at **run time**

Picks **WHERE** the algorithm will run

Does not compile anything for that place

Function specifier

`__host__` / `__device__`

Decided at **compile time**

Picks **WHAT** the function is compiled for

Both = a CPU version and a GPU version

Kitchen analogy

The Function Specifier is which kitchens the recipe is written for. The policy is which kitchen you send the order to.

Send an order to a kitchen that has no recipe for it, and it fails before you even start (a compile error).

Which combinations work?

	lambda / function specifier		
policy	<code>__host__</code>	<code>__device__</code>	<code>__host__ __device__</code>
<code>thrust::host</code>	runs on CPU	compile error	runs on CPU
<code>thrust::device</code>	compile error	runs on GPU	runs on GPU

Rule of thumb: mark lambdas `__host__ __device__` and they work with either policy.

Exercise 1: where does each line run?

5 minutes · replace each "???" with CPU or GPU

```
1  cv::where_am_I("???");
2
3  thrust::universal_vector<int> vec{1};
4  thrust::for_each(thrust::device, vec.begin(), vec.end(),
5      [] __host__ __device__(int) { cv::where_am_I("???"); });
6
7  thrust::for_each(thrust::host, vec.begin(), vec.end(),
8      [] __host__ __device__(int) { cv::where_am_I("???"); });
9
10 cv::where_am_I("???");
```

How to check

Each where_am_I call prints
[OK] or [MISS].

Run:

```
make
ex1_annotate_problem
./ex1_annotate_problem
```

File: 01-execution-spaces/ex1_annotate_problem.cu

Exercise 1: solution

```
1  cv::where_am_I("CPU");
2
3  thrust::universal_vector<int> vec{3};
4  thrust::for_each(thrust::device, vec.begin(), vec.end(),
5      [] __host__ __device__(int) { cv::where_am_I("GPU"); });
6
7  thrust::for_each(thrust::host, vec.begin(), vec.end(),
8      [] __host__ __device__(int) { cv::where_am_I("CPU"); });
9
10 cv::where_am_I("CPU");
```

Line 1

main() always starts on the CPU.

Line 5

thrust::device + a lambda that has a GPU version → GPU.

Line 8

Same lambda, thrust::host → CPU.

Line 10

Back on the CPU. Device execution does not "leak" out of the algorithm.

Key idea: the policy only affects the algorithm it is passed to. Everything else stays on the CPU.

Exercise 2: move it to the GPU

5 minutes · this loop prints on the CPU. Make it run on the GPU.

```
thrust::for_each(thrust::host, vec.begin(), vec.end(), [] __host__ (int val) {  
    printf("printing %d on %s\n", val, cv::execution_space());  
});
```

Hint

You need to change TWO words. One controls where it runs, the other controls what gets compiled.

Expected output

```
printing 1 on GPU  
printing 2 on GPU  
printing 3 on GPU
```

File: 01-execution-spaces/ex2_change_space_problem.cu

Exercise 2: solution

Before: CPU

```
thrust::for_each(thrust::host, vec.begin(), vec.end(), [] __host__ (int val) {  
    printf("printing %d on %s\n", val, cv::execution_space());  
});
```

After: GPU

```
thrust::for_each(thrust::device, vec.begin(), vec.end(), [] __device__ (int val) {  
    printf("printing %d on %s\n", val, cv::execution_space());  
});
```

thrust::device

Policy: run this algorithm on the GPU.

__device__

Specifier: compile this lambda for the GPU. (`__host__` `__device__` also works.)

Exercise 3: port std:: code to the GPU

A cooling simulation: each cell moves halfway toward room temperature every step.

Problem (CPU, std::)

```
std::vector<float> temp{42, 24, 50};

auto op = [=](float t) {
    return t + k * (room - t);
};

std::transform(temp.begin(), temp.end(),
               temp.begin(), op);
```

Solution (GPU, thrust::)

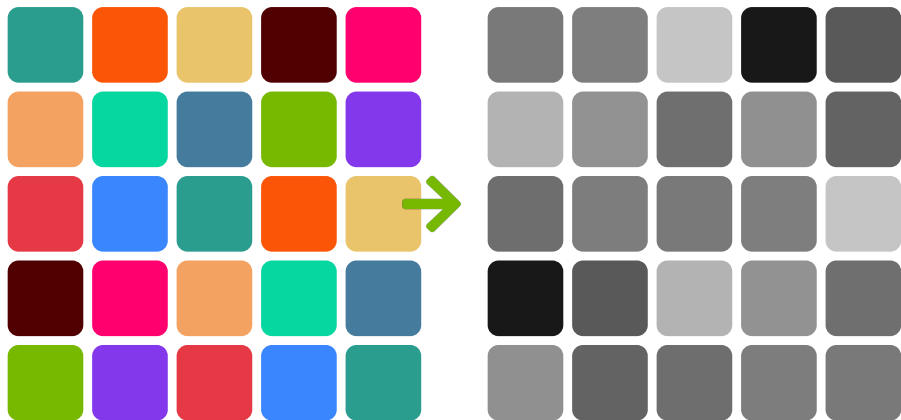
```
thrust::universal_vector<float> temp{42, 24, 50};

auto op = [=] __host__ __device__(float t) {
    return t + k * (room - t);
};

thrust::transform(thrust::device,
                  temp.begin(), temp.end(), temp.begin(), op);
```

- `std::vector` → `universal_vector` (CPU and GPU can both read it)
- Mark the lambda `__host__ __device__`
- `std::transform` → `thrust::transform` + `thrust::device`

Potential Project 1: color to grayscale



Input: RGB

Output: gray

$$\text{gray} = 0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$$

- Each thread reads one pixel's red, green and blue values.
- It mixes them into one brightness value.
- It writes that single gray pixel back out.

~2 million threads for one 1080p photo (1920×1080 pixels), all running in parallel.
Getting use of GPU performance vs the CPU.

The road ahead



Setup

Free cloud GPU and our first Thrust program.



Thrust

Host vs device, `std::` to `thrust::` (today).



Grayscale

Project 1:
`thrust::transform` over every pixel.



Level up

Write our own kernels:
blur, edges, and beyond.



What you need to join

A laptop, a Google account, and curiosity.